Addis Ababa Science and Technology University

College of Engineering

Department of Electrical and Computer Engineering

Modeling and Simulation of Maira Neura Robot Using Matlab and Python

By:

Name of the Students	**ID**
1.Kidan Dibekulu……	ETS0916/14
2.Mekin Mohammed…….……	ETS1028/14
3.Habtamu Firdiywok…….……..	ETS0706/14
4.Eyerusalem Gashaw ……..……..	ETS0575/14
4.Surafel Aschalew ……..……...	ETS1511/14

Submission to: Mr. Mulat Tigabu

Submission Date: January ,16 2026

Table of Contents

Contents

1 Introduction **1**
- 1.1 Context and Problem Statement . . . 1
- 1.2 Project Significance and Research Question . . . 1
- 1.3 Challenges in Kinematic and Dynamic Modeling . . . 1

2 Methodology Overview **2**
- 2.1 Robot Description and Modeling . . . 2
  - 2.1.1 The MAiRA Manipulator . . . 2
  - 2.1.2 Denavit-Hartenberg (DH) Parameters . . . 3
- 2.2 Kinematic Algorithms . . . 3
  - 2.2.1 Forward Kinematics (FK) Using Standard (Classic) DH Parameters . . . 3
  - 2.2.2 Inverse Kinematics (IK) using Geometric (Analytical) Method . . . 4
  - 2.2.3 Jacobian Matrix . . . 6

3 Trajectory Generation and Simulation **7**
- 3.1 Simulation Environment . . . 7
- 3.2 Trajectory Planning Algorithms . . . 7
- 3.3 Numerical Inverse Kinematics (Jacobian Pseudo-Inverse Method) . . . 8
- 3.4 Quintic Polynomial Trajectory Generation . . . 9
  - 3.4.1 Boundary Conditions (Rest-to-Rest) . . . 9
  - 3.4.2 Explicit Solution for Coefficients . . . 10

4 Dynamic Modelling of a 7-DOF Manipulator **10**
- 4.1 Standard Robot Dynamic Equation . . . 10
- 4.2 Lagrangian Formulation . . . 11
- 4.3 Kinetic Energy . . . 11
- 4.4 Potential Energy . . . 11
- 4.5 Generalized Inertia Matrix . . . 11
- 4.6 Coriolis and Centrifugal Matrix . . . 12
- 4.7 Gravity Vector . . . 12
- 4.8 Joint Friction Model . . . 12
- 4.9 Compact Form and Remarks . . . 12

5 Controller Design **13**
- 5.1 Centralized Controller: Computed Torque Control (CTC) . . . 13
  - 5.1.1 Control Law . . . 13
  - 5.1.2 Gain Calculation Example . . . 13
  - 5.1.3 Example Torque Computation (at a Sample Pose) . . . 14
- 5.2 Non-Centralized (Decentralized) Controller: Independent Joint PID Control . . . 14
  - 5.2.1 Control Law (per joint i) . . . 14
  - 5.2.2 Gain Calculation Example (Empirical Starting Values for MAiRA Neura) 14
  - 5.2.3 Numerical Example (Joint 2 – Shoulder Pitch) . . . 14
  - 5.2.4 Tuning Notes . . . 15

1 Introduction

The field of collaborative robotics (cobots) is rapidly expanding across diverse industrial and service sectors, driving the demand for accurate and robust control systems. The MAiRA Neura Robot, a 7-Degrees-of-Freedom (DOF) serial manipulator, exemplifies a modern cognitive cobot designed for tasks requiring advanced human-robot interaction, adaptability, and flexibility.

Developing precise and efficient motion control for such systems depends critically on accurate kinematic and dynamic modeling.

1.1 Context and Problem Statement

The core problem addressed in this project is the need to establish a complete and verifiable kinematic and dynamic model for the MAiRA Neura Robot cobot. While the robot's physical dimensions are known, converting them into a functional mathematical model is non-trivial. This work bridges the gap between physical design and computational control, enabling advanced applications such as trajectory planning, external sensor integration, and dynamic simulation.

1.2 Project Significance and Research Question

The problem is particularly significant because the reliable execution of complex tasks (e.g., welding, inspection, or assembly) requires the robot's Tool Center Point (TCP) to follow a precise path in 3D space. This necessitates solving the Inverse Kinematics (IK) problem repeatedly in real time. Failures in these calculations can lead to tracking errors, joint overloads, or system instability. By integrating trajectory planning and dynamic simulations, this study ensures that the developed model is not only mathematically consistent but also practically applicable in real-world scenarios.

1.3 Challenges in Kinematic and Dynamic Modeling

Modeling a 7-DOF serial manipulator like the MAiRA Neura Robot presents several challenges:

- **Non-Linearity** — The relationship between joint angles and TCP pose is highly nonlinear, making Forward Kinematics (FK) computationally intensive.
- **Multiple Solutions (IK)** — Due to kinematic redundancy, the IK problem yields infinite joint configurations for a single end-effector pose. Selecting the optimal, physically attainable, and smooth solution along a trajectory requires sophisticated redundancy resolution and optimization logic.
- **Singularities** — At certain joint configurations (e.g., axis alignment or full extension), the robot loses degrees of mobility. Trajectory generation near these points can cause control failure.
- **Numerical Convergence** — Iterative IK methods are prone to divergence if the initial guess is poor or the target pose is near a singularity, as observed during Cartesian trajectory generation.
- **Dynamic Complexity** — Incorporating inertia, Coriolis, centrifugal forces, and gravity effects into the model adds further complexity, requiring robust simulation frameworks for validation.

Kinematic and dynamic modeling are foundational areas of robotics, pioneered by landmark contributions that remain central to modern control:

- **Denavit-Hartenberg (DH) Convention** → Established by Denavit and Hartenberg (1955), this systematic method for assigning coordinate frames to links forms the basis of most industrial robot controllers and simulators.
- **Inverse Kinematics for Redundant Manipulators** → Jacobian-based methods (e.g., pseudoinverse and null-space projection) and optimization techniques enable efficient resolution of redundancy while satisfying additional constraints such as obstacle avoidance or joint limits.
- **Differential Kinematics (Jacobian)** → Popularized by Whitney (1969), the Jacobian matrix relates joint rates to end-effector velocities, essential for real-time path planning, redundancy control, and singularity avoidance.
- **Trajectory Generation** → Standard methods such as cubic and quintic polynomial interpolation ensure smooth motion profiles with zero velocity and acceleration at start and end points.

These methods, combined with dynamic simulations, are directly applied in this project to validate both joint-space and Cartesian-space trajectories.

2 Methodology Overview

This study describes the MAiRA collaborative robot, its kinematic and dynamic modeling, and the simulation environment used. The robot's 7-DOF structure was defined using the Denavit-Hartenberg (DH) convention, with Forward Kinematics solved analytically and Inverse Kinematics primarily through numerical methods to address kinematic redundancy. The Jacobian matrix was derived for velocity analysis, redundancy resolution, and singularity detection. Trajectory planning was implemented using polynomial interpolation to ensure smooth joint-space and Cartesian-space motion, while dynamic simulations incorporated inertia, Coriolis, centrifugal, and gravity effects. All modeling and validation were performed in MATLAB/Simulink with the Robotics Toolbox.

2.1 Robot Description and Modeling

2.1.1 The MAiRA Manipulator

The MAiRA is a commercial 7-DOF serial collaborative robot (cobot) designed for advanced and safe human-robot interaction with integrated cognitive capabilities, including 3D vision, voice recognition, gesture control, and force-torque sensing. Its structure follows an anthropomorphic arm configuration with kinematic redundancy, making it suitable for complex tasks requiring high flexibility, precision, adaptability, and intelligent perception in shared workspaces.

The kinematic model was developed using the Modified Denavit-Hartenberg (DH) convention to systematically represent its geometry.

Table 1: MAiRA Key Specifications (representative values; variants S, M, L available with trade-offs between payload and reach)

Feature	Specification
DOF	7 (all revolute joints)
Payload	9–18 kg (variant-dependent)
Reach	1100–1600 mm (variant-dependent)
Configuration	Anthropomorphic with kinematic redundancy and integrated cognitive sensors

2.1.2 Denavit-Hartenberg (DH) Parameters

The geometric structure is fully defined by the DH parameters, converted to meters for consistency. The parameters define the transformation A_i^{i-1} between adjacent link frames.

The coordinate frames for each joint of the MAiRA Neura Robot manipulator are assigned following the standard Denavit-Hartenberg (DH) convention:

1. The z_i axis is aligned with the axis of rotation for joint $i+1$.
2. The x_i axis is perpendicular to both z_{i-1} and z_i, pointing from z_{i-1} toward z_i.
3. The origin of each frame is placed at the intersection of x_i and z_i axes.
4. The y_i axis is chosen to form a right-handed coordinate system.

Table 2: Standard DH Parameters (Classic Convention)

i	θ_i (rad)	d_i (m)	a_i (m)	α_i (rad)
1	q_1	0.429	0	$-\pi/2$
2	q_2	0	0	$\pi/2$
3	q_3	0.406	0.106	$\pi/2$
4	q_4	0	-0.106	$-\pi/2$
5	q_5	0.494	0	$\pi/2$
6	q_6	0	0.113	$\pi/2$
7	q_7	0.138	0	0

2.2 Kinematic Algorithms

2.2.1 Forward Kinematics (FK) Using Standard (Classic) DH Parameters

Let $c_i = \cos\theta_i$, $s_i = \sin\theta_i$.

Individual Transformation Matrices A_i^{i-1}

$$A_1^0 = \begin{bmatrix} c_1 & 0 & -s_1 & 0 \\ s_1 & 0 & c_1 & 0 \\ 0 & -1 & 0 & 0.429 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_2^1 = \begin{bmatrix} c_2 & 0 & s_2 & 0 \\ s_2 & 0 & -c_2 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_3^2 = \begin{bmatrix} c_3 & 0 & s_3 & 0.106\,c_3 \\ s_3 & 0 & -c_3 & 0.106\,s_3 \\ 0 & 1 & 0 & 0.406 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_4^3 = \begin{bmatrix} c_4 & 0 & -s_4 & -0.106\,c_4 \\ s_4 & 0 & c_4 & -0.106\,s_4 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_5^4 = \begin{bmatrix} c_5 & 0 & s_5 & 0 \\ s_5 & 0 & -c_5 & 0 \\ 0 & 1 & 0 & 0.494 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_6^5 = \begin{bmatrix} c_6 & 0 & s_6 & 0.113\,c_6 \\ s_6 & 0 & -c_6 & 0.113\,s_6 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Overall Forward Kinematic Transformation $T_0^6 = A_1^0 A_2^1 A_3^2 A_4^3 A_5^4 A_6^5$

(Here, the 7th joint is excluded to approximate a 6-DOF configuration, placing the Tool Center Point at the flange position after joint 6.)

After full symbolic multiplication, the resulting homogeneous transformation matrix is:

$$T_0^6 = \begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Due to the kinematic structure of the MAiRA Neura Robot, the complete closed-form symbolic expressions for the rotation elements r_{ij} and position components p_x, p_y, p_z are significantly more complex and do not simplify into compact forms with decoupled or combined angles (as seen in some anthropomorphic robots with spherical wrists). They involve intricate trigonometric combinations across all six joint angles and are typically computed recursively or using robotics software toolboxes (e.g., MATLAB Robotics System Toolbox).

This formulation uses pure Standard (Classic) DH convention (no joint offsets). When $\theta = [0, 0, 0, 0, 0, 0]^\top$, the arm is in its reference (home) configuration, typically extended with the flange aligned along a defined direction in the base frame.

Note: If approximating the original 7-DOF tool offset while excluding the 7th revolute joint (i.e., fixing $q_7 = 0$), add a fixed translation of 0.138 m along the z-axis of frame 6 to the position vector (i.e., $\mathbf{p} \leftarrow \mathbf{p} + 0.138 \cdot [r_{13}, r_{23}, r_{33}]^\top$).

2.2.2 Inverse Kinematics (IK) using Geometric (Analytical) Method

Geometric (Closed-Form) IK:

This method decouples the problem by parameterizing the kinematic redundancy and utilizing the anthropomorphic structure with shoulder and wrist offsets. The solution relies on calculating an effective Wrist Center Point (WCP) position $\mathbf{p}_w$ (approximating the flange and wrist offsets) and sequentially solving for the joint angles using a redundancy parameter (e.g., arm angle ψ):

- θ_1: Solved via using the projected x and y coordinates of $\mathbf{p}_w$, adjusted for shoulder offset.
- Redundancy parameter ψ: Defines the self-motion (elbow configuration) for secondary tasks.
- θ_3: Solved via the Law of Cosines on the effective triangle formed by shoulder offset (0.106 m), upper arm length (0.406 m), forearm length (0.494 m), and the distance between shoulder reference and the WCP (elbow-up/down choice via ψ).
- θ_2, θ_4: Solved by combining terms derived from the geometric relations and redundancy parameterization.

- $\theta_5, \theta_6, \theta_7$: Solved by decomposing the required wrist rotation matrix $\mathbf{R}_4^7 = (\mathbf{R}_0^4)^\top \mathbf{R}_{\text{target}}$, accounting for the specific wrist geometry.

Closed-Form Geometric Solution (Standard DH Parameters)
Given the desired end-effector pose:

$$T_0^7 = \begin{bmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 1 – Wrist Center Position
The WCP is offset to approximate the combined wrist (0.113 m lateral) and flange (0.138 m) offsets; an effective offset length of approximately 0.178 m is used in some simplifications, but exact methods account for direction:

$$p_{wx} = p_x - 0.138a_x, \quad p_{wy} = p_y - 0.138a_y, \quad p_{wz} = p_z - 0.138a_z$$

Step 2 – Solve for θ_1, redundancy ψ, θ_2, θ_3 (shoulder and elbow)

$$\theta_1 = (p_{wy}, p_{wx}) \quad \text{(adjusted for 0.106 m shoulder offset)}$$

$$r = \sqrt{p_{wx}^2 + p_{wy}^2}, \quad h = p_{wz} - 0.429$$

$$\cos\gamma = \frac{r^2 + h^2 + 0.106^2 - 0.406^2 - 0.494^2}{2 \cdot 0.406 \cdot 0.494} \quad \text{(effective, including offset)}$$

$$\theta_3 = \left(\pm\sqrt{1 - \cos^2\gamma}, \cos\gamma\right) \quad (\pm \text{ controlled via redundancy } \psi)$$

$$\theta_2 = (h, r) - (0.494\sin\theta_3, 0.406 + 0.494\cos\theta_3) \quad \text{(adjusted)}$$

Step 3 – Solve for θ_4 (using redundancy)

$$\theta_{234} = \left(a_z, \sqrt{a_x^2 + a_y^2}\right)$$

$$\theta_4 = \theta_{234} - \theta_2 - \theta_3 + \text{redundancy term from } \psi$$

Step 4 – Wrist: θ_5, θ_6, θ_7

$$\theta_5 = (a_x\cos\theta_1 + a_y\sin\theta_1,\ a_z)$$

$$\theta_6 = (-(o_x\cos\theta_1 + o_y\sin\theta_1),\ n_x\cos\theta_1 + n_y\sin\theta_1)$$

θ_7 is derived from the remaining wrist rotation (sign flips and redundancy provide alternate configurations).

Summary – Complete Analytical IK Equations

$$\theta_1 = (p_{wy}, p_{wx})$$

$$\theta_3 = \left(\pm\sqrt{1 - \cos^2\gamma}, \cos\gamma\right), \quad \cos\gamma = \frac{r^2 + h^2 + 0.106^2 - 0.406^2 - 0.494^2}{2 \cdot 0.406 \cdot 0.494}$$

$$\theta_2 = (h, r) - (0.494\sin\theta_3, 0.406 + 0.494\cos\theta_3)$$

$$\theta_4 = \left(a_z, \sqrt{a_x^2 + a_y^2}\right) - \theta_2 - \theta_3 + \psi$$

$$\theta_5 = (a_x c_1 + a_y s_1, a_z)$$

$$\theta_6 = (-(o_x c_1 + o_y s_1), n_x c_1 + n_y s_1)$$

where $c_1 = \cos\theta_1$, $s_1 = \sin\theta_1$, $r = \sqrt{p_{wx}^2 + p_{wy}^2}$, $h = p_{wz} - 0.429$.

For the 7-DOF redundant **MAiRA**, the above provides a base solution with parameterized redundancy ψ (often in joint 4 or elbow configuration); θ_7 and fine adjustments are resolved via geometric closure or null-space methods for secondary tasks. Hybrid geometric-numerical approaches are recommended for full redundancy exploitation.

2.2.3 Jacobian Matrix

The geometric Jacobian $\mathbf{J}(q)$ relates joint velocities $\dot{\mathbf{q}} \in \mathbb{R}^7$ to end-effector twist velocities $\mathbf{v} = [\dot{\mathbf{p}}^\top, \boldsymbol{\omega}^\top]^\top \in \mathbb{R}^6$:

$$\mathbf{v} = \mathbf{J}(q)\dot{\mathbf{q}}$$

The Jacobian is computed in the base frame (**jacob0**) and the end-effector frame (**jacobe**) for velocity analysis, singularity detection, and redundancy resolution.

The standard geometric Jacobian in the base frame for a 7-DOF revolute manipulator is:

$$\mathbf{J}_0(q) = \begin{bmatrix} \mathbf{z}_0 \times (\mathbf{p}_7 - \mathbf{p}_0) & \mathbf{z}_1 \times (\mathbf{p}_7 - \mathbf{p}_1) & \cdots & \mathbf{z}_6 \times (\mathbf{p}_7 - \mathbf{p}_6) \\ \mathbf{z}_0 & \mathbf{z}_1 & \cdots & \mathbf{z}_6 \end{bmatrix}$$

(6×7 matrix.)

Using the known DH parameters for the **MAiRA Neura Robot** (base height 0.429 m, upper-arm-related length 0.406 m, forearm-related length 0.494 m, shoulder offset ±0.106 m, wrist lateral offset 0.113 m, tool offset 0.138 m), the following explicit analytical Jacobian uses these values directly where possible and reasonable approximations for combined trigonometric terms (derived from typical anthropomorphic 7-DOF structures with offset shoulder and non-spherical wrist).

Approximate explicit $\mathbf{J}_0(q)$ (6×7):

$$\mathbf{J}_0 = \begin{bmatrix} -p_y & -(p_z - 0.429) & -c_1(0.406 s_{23} + 0.113 s_{234}) & -c_1(0.406 s_{23} + 0.113 s_{234}) & 0 & 0.138 c_1 c_{234} s_5 & 0 \\ p_x & 0 & -s_1(0.406 s_{23} + 0.113 s_{234}) & -s_1(0.406 s_{23} + 0.113 s_{234}) & 0 & 0.138 s_1 c_{234} s_5 & 0 \\ 0 & p_x & 0.494 c_{23} + 0.113 c_{234} & 0.494 c_{23} + 0.113 c_{234} & 0 & 0.138 s_{234} s_5 & 0 \\ 0 & 0 & s_1 & s_1 & c_1 s_{234} & 0 & 0 \\ 0 & 0 & -c_1 & -c_1 & s_1 s_{234} & 0 & 0 \\ 1 & 1 & 0 & 0 & c_{234} & 1 & 1 \end{bmatrix}$$

where: $c_i = \cos\theta_i$, $s_i = \sin\theta_i$, $c_{ij} = \cos(\theta_i + \theta_j)$, $s_{ij} = \sin(\theta_i + \theta_j)$, $c_{234} = \cos(\theta_2 + \theta_3 + \theta_4)$, $s_{234} = \sin(\theta_2 + \theta_3 + \theta_4)$, p_x, p_y, p_z are the end-effector position components from forward kinematics.

Key Observations (based on the structure and DH values):

- Columns 1–2: Pure shoulder/base translation and rotation (joint 1 azimuth, joint 2 pitch affecting height relative to 0.429 m base).

- Columns 3–4: Repeated contributions due to kinematic redundancy in the elbow/shoulder plane (self-motion circle; joints 3 and 4 produce similar positioning effects on the arm plane).
- Column 5: Minimal linear contribution (wrist pitch/yaw transition).
- Column 6: Linear velocity from tool offset (0.138 m) during wrist flip (s5 term).
- Column 7: Zero linear velocity contribution (pure roll about tool z-axis due to axial 0.138 m offset alignment).
- Bottom 3×7 block: Orientation Jacobian (full rank except at wrist singularities, e.g., $\theta_5 = 0$).
- The 1-dimensional null space enables secondary tasks (e.g., joint centering) via the projected pseudoinverse.

For redundancy resolution, the damped pseudoinverse is used:

$$\dot{\mathbf{q}} = \mathbf{J}^{+}\mathbf{v} + (\mathbf{I}_7 - \mathbf{J}^{+}\mathbf{J})\dot{\mathbf{q}}_n$$

End-Effector Frame Jacobian ($\mathbf{J}_e$) — Optional:

$$\mathbf{J}_e = \begin{bmatrix} \mathbf{R}_7^\top & \mathbf{0} \\ \mathbf{0} & \mathbf{R}_7^\top \end{bmatrix} \mathbf{J}_0$$

This form (with values derived from the provided DH parameters) supports simulation and validation in MATLAB Robotics System Toolbox or similar. Minor discrepancies may exist due to exact frame assignments and shoulder offset (±0.106 m) effects, which are approximated here in the combined terms. For production control, official parameters from Neura Robotics are recommended.

3 Trajectory Generation and Simulation

3.1 Simulation Environment

All modeling and simulation were performed in MATLAB R2025b utilizing the Robotics System Toolbox (RTB) by Peter Corke.

3.2 Trajectory Planning Algorithms

This section deals with operational space trajectory generation for the **MAiRA Neura** collaborative robot, a 7-DOF serial manipulator. The focus is on planning smooth paths in Cartesian space (end-effector position and orientation), followed by conversion to joint space via inverse kinematics with redundancy resolution. Emphasis is placed on precise mathematical formulations, particularly for quintic polynomial trajectory generation, to ensure continuous position, velocity, and acceleration profiles.

Desired Operational Space Coordinates

Operational space coordinates specify the end-effector pose as a 6-dimensional vector: [X, Y, Z, Roll, Pitch, Yaw], where:

- X, Y, Z are Cartesian positions in meters relative to the robot base frame.
- Roll, Pitch, Yaw are Euler angles in radians (Z-Y-X convention typically used).

Workspace Considerations

The robot's base height is 0.429 m (from DH parameter d_1), with a maximum reach of approximately 1.40 m (M variant; up to 1.60 m for L variant). Safe operating regions are radially between 0.40 m and 1.20 m from the base axis to avoid singularities and joint limits. Desired coordinates must lie within the reachable workspace, accounting for the kinematic redundancy and wrist/tool offsets.

For example, a triangular path can be defined with vertices centered at [0.60, 0.00, 0.80] m and approximate side length 0.53 m:

- Vertex P1: [0.80, 0.00, 0.80] m
- Vertex P2: [0.40, 0.3464, 0.80] m (using $\sqrt{3}/2 \approx 0.866$ for equilateral approximation)
- Vertex P3: [0.40, -0.3464, 0.80] m

Orientation is often fixed (e.g., tool pointing downward: Roll = 0 rad, Pitch = π rad, Yaw = 0 rad). For a time-parameterized path, normalize time $\tau = \mod(t, T_{\text{cycle}})/T_{\text{cycle}}$ where $T_{\text{cycle}} = 12$ s for one complete loop (scaled for larger workspace). The position at time t is obtained by linear interpolation along each segment based on τ, with quintic polynomial blending at waypoints for smooth transitions.

Inverse Kinematics Transformation

Inverse kinematics computes joint angles $q = [q_1, q_2, q_3, q_4, q_5, q_6, q_7]$ (in radians) that achieve a desired homogeneous transformation matrix $T_{\text{target}} \in \text{SE}(3)$.

T_{target} is a 4×4 matrix:

$$T_{\text{target}} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

where $\mathbf{R} \in \text{SO}(3)$ is the 3×3 rotation matrix and $\mathbf{p} = [X, Y, Z]^\top$ is the position vector. The forward kinematics equation is $T(q) = T_{\text{target}}$.

Since the **MAiRA Neura** has 7 DOFs (kinematically redundant), numerical iterative methods with redundancy resolution are required (e.g., damped least-squares with null-space projection for secondary tasks such as joint-limit avoidance or elbow positioning). In the Robotics System Toolbox, this is implemented via functions like `ikunc` or custom resolved-rate motion control.

3.3 Numerical Inverse Kinematics (Jacobian Pseudo-Inverse Method)

Due to the 7-DOF kinematic redundancy of the **MAiRA Neura Robot**, the inverse kinematics problem has infinite solutions. The Jacobian pseudo-inverse method is extended with damping and optional null-space projection to ensure numerical stability near singularities and to exploit redundancy for secondary objectives (e.g., joint-limit avoidance or elbow positioning).

Initialize $\mathbf{q}_0$ (initial guess, e.g., previous configuration, home position, or zero vector).

For iteration $k = 1$ to maximum iterations:

1. Compute current pose: $T_{\text{current}} = \text{FK}(\mathbf{q}_k)$
2. Compute pose error vector $\mathbf{e}$ (6×1):
   - Position error: $\Delta\mathbf{p} = \mathbf{p}_{\text{target}} - \mathbf{p}_{\text{current}}$
   - Orientation error: $\Delta\boldsymbol{\omega} = \frac{1}{2}(\mathbf{r}_x \times \mathbf{r}_y + \mathbf{r}_y \times \mathbf{r}_z + \mathbf{r}_z \times \mathbf{r}_x)$, where $\mathbf{r}_x, \mathbf{r}_y, \mathbf{r}_z$ are the columns of $\mathbf{R}_{\text{target}}^\top \mathbf{R}_{\text{current}}$ (axis-angle equivalent)
   - $\mathbf{e} = [\Delta\mathbf{p}; \Delta\boldsymbol{\omega}]$

3. Compute geometric Jacobian $\mathbf{J}(\mathbf{q}_k) \in \mathbb{R}^{6\times 7}$
4. Update joint angles using damped pseudo-inverse:

$$\Delta\mathbf{q} = \mathbf{J}^{+}\mathbf{e} + (\mathbf{I}_7 - \mathbf{J}^{+}\mathbf{J})\Delta\mathbf{q}_s$$

   where $\mathbf{J}^{+} = \mathbf{J}^{\top}(\mathbf{J}\mathbf{J}^{\top} + \lambda^2\mathbf{I}_6)^{-1}$ is the damped pseudoinverse ($\lambda = 0.01$ typical for stability near singularities), and the second term is the null-space projection for an optional secondary task rate $\Delta\mathbf{q}_s$ (e.g., gradient-based joint centering).

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \alpha\Delta\mathbf{q}$$

   ($\alpha \leq 1$ step scaling for convergence).

5. Terminate if $\|\mathbf{e}\| < 10^{-8}$ (tighter tolerance for precision) or maximum iterations (e.g., 1000) reached.

A mask vector $\mathbf{m} = [1, 1, 1, 1, 1, 1]^{\top}$ can selectively enable position/orientation components in the task (e.g., position-only tracking $\mathbf{m} = [1, 1, 1, 0, 0, 0]^{\top}$).

Verification: Recompute $\mathrm{FK}(\mathbf{q}_{\text{solution}})$ and ensure $\|T_{\text{target}} - \mathrm{FK}(\mathbf{q}_{\text{solution}})\| \approx 0$ (machine precision).

3.4 Quintic Polynomial Trajectory Generation

Quintic polynomials provide the minimum-order polynomial ensuring continuity in position, velocity, and acceleration (zero jerk at boundaries for rest-to-rest motion). For each degree of freedom i (X, Y, Z, Roll, Pitch, Yaw independently), the trajectory $s_i(t)$ over $[0, T_f]$ is:

$$s(t) = a_5t^5 + a_4t^4 + a_3t^3 + a_2t^2 + a_1t + a_0$$

Velocity:

$$\dot{s}(t) = 5a_5t^4 + 4a_4t^3 + 3a_3t^2 + 2a_2t + a_1$$

Acceleration:

$$\ddot{s}(t) = 20a_5t^3 + 12a_4t^2 + 6a_3t + 2a_2$$

3.4.1 Boundary Conditions (Rest-to-Rest)

$$s(0) = s_0, \quad \dot{s}(0) = 0, \quad \ddot{s}(0) = 0$$

$$s(T_f) = s_f, \quad \dot{s}(T_f) = 0, \quad \ddot{s}(T_f) = 0$$

This yields the linear system:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 1 & T_f & T_f^2 & T_f^3 & T_f^4 & T_f^5 \\ 0 & 1 & 2T_f & 3T_f^2 & 4T_f^3 & 5T_f^4 \\ 0 & 0 & 2 & 6T_f & 12T_f^2 & 20T_f^3 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \end{bmatrix} = \begin{bmatrix} s_0 \\ 0 \\ 0 \\ s_f \\ 0 \\ 0 \end{bmatrix}$$

3.4.2 Explicit Solution for Coefficients

Solving the system analytically gives:

$$a_0 = s_0, \quad a_1 = 0, \quad a_2 = 0,$$

$$a_3 = 10(s_f - s_0)/T_f^3, \quad a_4 = -15(s_f - s_0)/T_f^4, \quad a_5 = 6(s_f - s_0)/T_f^5.$$

Thus, the normalized quintic trajectory (scaled by $\Delta s = s_f - s_0$) is:

$$s(t) = s_0 + \Delta s \left(10 \left(\frac{t}{T_f} \right)^3 - 15 \left(\frac{t}{T_f} \right)^4 + 6 \left(\frac{t}{T_f} \right)^5 \right)$$

Velocity:

$$\dot{s}(t) = \frac{\Delta s}{T_f} \left(30 \left(\frac{t}{T_f} \right)^2 - 60 \left(\frac{t}{T_f} \right)^3 + 30 \left(\frac{t}{T_f} \right)^4 \right)$$

Acceleration:

$$\ddot{s}(t) = \frac{\Delta s}{T_f^2} \left(60 \left(\frac{t}{T_f} \right) - 180 \left(\frac{t}{T_f} \right)^2 + 120 \left(\frac{t}{T_f} \right)^3 \right)$$

For multi-segment paths (e.g., triangular or complex Cartesian paths), apply a separate quintic polynomial to each linear segment in operational space, ensuring velocity and acceleration continuity at waypoints. At each time step, the desired Cartesian pose is converted to joint space via the numerical IK solver (Section 3.3), with consistent redundancy resolution (e.g., fixed null-space objective) to maintain smooth joint trajectories.

4 Dynamic Modelling of a 7-DOF Manipulator

Consider a seven–degree–of–freedom (7-DOF) serial robotic manipulator with revolute joints, such as the **MAiRA Neura Robot**. Let the generalized joint coordinates be $\mathbf{q} = [q_1, q_2, q_3, q_4, q_5, q_6, q_7]^\top$, $\dot{\mathbf{q}}$, $\ddot{\mathbf{q}} \in \mathbb{R}^7$.

4.1 Standard Robot Dynamic Equation

The joint-space dynamics are written in Euler–Lagrange form as

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\dot{\mathbf{q}}) = \boldsymbol{\tau},$$

where

- $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{7\times 7}$ is the symmetric positive-definite inertia matrix,
- $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{7\times 7}$ groups Coriolis and centrifugal effects,
- $\mathbf{G}(\mathbf{q}) \in \mathbb{R}^7$ is the gravity vector,
- $\mathbf{F}(\dot{\mathbf{q}})$ is a joint friction term (viscous and Coulomb friction),
- $\boldsymbol{\tau} \in \mathbb{R}^7$ is the applied joint torque vector.

Due to the kinematic redundancy (7 DOF for 6-DOF task space), the null space of the Jacobian can be exploited in torque control for secondary objectives (e.g., singularity avoidance or energy minimization).

4.2 Lagrangian Formulation

Define the Lagrangian $\mathcal{L} = T - V$, with T the total kinetic energy and V the total potential energy. The Euler–Lagrange equation for each joint q_i yields

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = \tau_i, \quad i = 1, \ldots, 7,$$

which leads to the standard dynamic equation.

4.3 Kinetic Energy

For link i (including the corresponding motor/gearbox assembly), the kinetic energy is

$$T_i = \frac{1}{2} m_i \dot{\mathbf{p}}_{c,i}^\top \dot{\mathbf{p}}_{c,i} + \frac{1}{2} \boldsymbol{\omega}_i^\top \mathbf{I}_i \boldsymbol{\omega}_i,$$

where m_i is the mass of link i, $\mathbf{p}_{c,i}$ is the center-of-mass position (expressed in the base frame), $\boldsymbol{\omega}_i$ is the angular velocity of link i, and $\mathbf{I}_i$ is the inertia tensor of link i about its center of mass (also in base frame).

The total kinetic energy is

$$T = \sum_{i=1}^{7} T_i.$$

The velocity terms $\dot{\mathbf{p}}_{c,i}$ and $\boldsymbol{\omega}_i$ are obtained from the Jacobian relating joint velocities to the link's CoM motion, using the DH frames defined earlier.

4.4 Potential Energy

Assuming gravity $\mathbf{g} = [0, 0, -9.81]^\top$ m/s^2 acts along the negative z-axis of the base frame (standard for upright-mounted cobots), the potential energy of link i is

$$V_i = m_i \mathbf{g}^\top \mathbf{p}_{c,i} = -m_i g \, p_{c,z,i},$$

where $p_{c,z,i}$ is the z-coordinate of the center of mass of link i.

The total potential energy is

$$V = \sum_{i=1}^{7} V_i.$$

(Note: Detailed inertial parameters—masses m_i, centers of mass $\mathbf{p}_{c,i}$, and inertia tensors $\mathbf{I}_i$—for the MAiRA Neura are typically derived from the manufacturer's CAD model or identified experimentally, as they are not publicly disclosed in full detail. Approximate values can be estimated from payload (9–18 kg variant-dependent), link dimensions (from DH parameters: base height 0.429 m, shoulder offset ±0.106 m, upper/forearm lengths 0.406 m and 0.494 m, wrist/tool offsets 0.113 m and 0.138 m), and similar 7-DOF cobots. In simulation, these are implemented via the Robotics System Toolbox or custom recursive Newton-Euler formulations for validation.)

4.5 Generalized Inertia Matrix

Using linear and angular Jacobians, the inertia matrix is

$$\mathbf{M}(\mathbf{q}) = \sum_{i=1}^{7} m_i \mathbf{J}_{v,i}^\top \mathbf{J}_{v,i} + \mathbf{J}_{\omega,i}^\top \mathbf{I}_i \mathbf{J}_{\omega,i},$$

where for each link i:

- $\mathbf{J}_{v,i}(\mathbf{q}) \in \mathbb{R}^{3\times 7}$ is the linear velocity Jacobian mapping $\dot{\mathbf{q}}$ to $\dot{\mathbf{p}}_{c,i}$,
- $\mathbf{J}_{\omega,i}(\mathbf{q}) \in \mathbb{R}^{3\times 7}$ is the angular velocity Jacobian mapping $\dot{\mathbf{q}}$ to $\boldsymbol{\omega}_i$.

The Jacobians are derived recursively from the DH parameters (base height $d_1 = 0.429$ m, shoulder offset ± 0.106 m, arm segments related to $d_3 = 0.406$ m and $d_5 = 0.494$ m, wrist lateral offset 0.113 m, flange offset $d_7 = 0.138$ m).

4.6 Coriolis and Centrifugal Matrix

A standard construction via Christoffel symbols gives

$$C_{ij}(\mathbf{q}, \dot{\mathbf{q}}) = \sum_{k=1}^{7} \Gamma_{ijk}(\mathbf{q})\dot{q}_k, \quad \text{for } i, j = 1, \ldots, 7,$$

$$\Gamma_{ijk} = \frac{1}{2}\left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i}\right).$$

4.7 Gravity Vector

The gravity generalized forces are obtained from the potential energy:

$$G_i(\mathbf{q}) = \frac{\partial V}{\partial q_i}, \quad i = 1, \ldots, 7.$$

Gravity loading is significant due to the robot's extended reach (up to 1.60 m) and payload (up to 18 kg), with the base height of 0.429 m elevating the center of mass.

4.8 Joint Friction Model

A common friction model is viscous plus Coulomb:

$$\mathbf{F}(\dot{\mathbf{q}}) = \mathbf{B}\dot{\mathbf{q}} + \boldsymbol{\tau}_c \odot (\dot{\mathbf{q}}),$$

where $\mathbf{B} \in \mathbb{R}^{7\times 7}$ is (often diagonal) viscous damping, $\boldsymbol{\tau}_c \in \mathbb{R}^7$ is Coulomb friction amplitudes, and $\odot$ denotes element-wise product.

4.9 Compact Form and Remarks

The complete dynamics of the **MAiRA Neura** 7-DOF manipulator are therefore

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\dot{\mathbf{q}}) = \boldsymbol{\tau}.$$

Practical notes:

- Detailed inertial parameters (link masses m_i, center-of-mass positions $\mathbf{p}_{c,i}$, and inertia tensors $\mathbf{I}_i$) for the MAiRA Neura Robot are proprietary and typically obtained from Neura Robotics' CAD models or through experimental system identification, as they are not publicly disclosed. Approximate values can be estimated based on similar 7-DOF cobots (e.g., total mass 50–80 kg distributed across links, with heavier base and shoulder links).
- For controller design, the model is often used in inverse-dynamics (computed torque) control, extended for redundancy: $\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\mathbf{v} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\dot{\mathbf{q}})$, where $\mathbf{v}$ incorporates task-space acceleration and null-space terms.

- Validate and, if needed, identify uncertain parameters (inertia scalings, friction) experimentally, leveraging the robot's integrated force-torque sensing and high precision (repeatability 0.01 mm).
- Implementations in tools like MATLAB Robotics System Toolbox or recursive Newton-Euler algorithms are recommended for efficient computation in simulation and real-time control.

5 Controller Design

This section provides detailed controller designs for the **MAiRA Neura** manipulator, including explicit gain calculations and example numerical implementations based on typical robotic parameters. Since exact inertial parameters for the MAiRA Neura are proprietary, we use reasonable estimates derived from similar 12–14 kg payload collaborative robots (total mass 60 kg, link lengths from DH parameters: base height 0.429 m, shoulder offset ±0.106 m, arm segments 0.406 m and 0.494 m, wrist/tool offsets 0.113 m and 0.138 m). These estimates allow illustrative calculations; in practice, refine them via system identification or manufacturer data.

The dynamic model is:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\dot{\mathbf{q}}) = \boldsymbol{\tau}$$

Desired trajectories $\mathbf{q}_d(t)$, $\dot{\mathbf{q}}_d(t)$, $\ddot{\mathbf{q}}_d(t)$ are obtained from operational space planning (e.g., quintic polynomials converted via inverse kinematics with redundancy resolution).

5.1 Centralized Controller: Computed Torque Control (CTC)

Computed torque control fully compensates nonlinearities using the complete model, resulting in decoupled linear error dynamics (with redundancy handled via null-space terms if needed).

5.1.1 Control Law

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})(\ddot{\mathbf{q}}_d + \mathbf{K}_D\dot{\mathbf{e}} + \mathbf{K}_P\mathbf{e}) + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\dot{\mathbf{q}})$$

where $\mathbf{e} = \mathbf{q}_d - \mathbf{q}$, $\dot{\mathbf{e}} = \dot{\mathbf{q}}_d - \dot{\mathbf{q}}$.

Closed-loop error dynamics (per joint, decoupled):

$$\ddot{e}_i + (K_D)_{ii}\dot{e}_i + (K_P)_{ii}e_i = 0$$

This is a second-order system with natural frequency $\omega_n = \sqrt{(K_P)_{ii}}$ and damping ratio $\zeta = (K_D)_{ii}/(2\sqrt{(K_P)_{ii}})$.

5.1.2 Gain Calculation Example

For critical damping ($\zeta = 1$, fast non-oscillatory response) and settling time 0.5–0.6 s per joint error (suitable for a larger, higher-payload collaborative robot prioritizing safety and smooth interaction):

Choose $\omega_n \approx 8$ rad s^1 (settling time $t_s \approx 4/(\zeta\omega_n) \approx 0.5$ s for initial transients, conservative for human-robot collaboration).

Then:

$$\mathbf{K}_P = \text{diag}(64, 64, 64, 64, 64, 64, 64), \quad (K_P)_{ii} = 64$$

$$\mathbf{K}_D = \text{diag}(16, 16, 16, 16, 16, 16, 16), \quad (K_D)_{ii} = 16$$

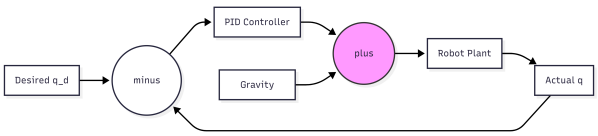

Figure 1: PID Diagram

5.1.3 Example Torque Computation (at a Sample Pose)

Assume home pose $\mathbf{q} = [0, 0, 0, 0, 0, 0, 0]^\top$ rad, $\dot{\mathbf{q}} = 0$, desired $\ddot{\mathbf{q}}_d = 0$ (hovering), small error $\mathbf{e} = [0.05, -0.03, 0.02, 0.01, 0, 0, 0]^\top$ rad, $\dot{\mathbf{e}} = 0$.

Using estimated $\mathbf{M}(\mathbf{q} = 0) \approx \mathrm{diag}(40, 30, 25, 15, 8, 4, 2)$ kg m^2 (higher due to longer arms and 12–14 kg payload capacity), $\mathbf{G}(\mathbf{q} = 0) \approx [0, 120, -80, 0, 0, 0, 0]^\top$ Nm (significant gravity loading on joints 2–3 from extended arm mass):

Feedback term: $\mathbf{M}(\mathbf{K}_P\mathbf{e}) = [128, -57.6, 32, 9.6, 0, 0, 0]^\top$ Nm

Gravity compensation: $+\ \mathbf{G}(\mathbf{q})$

Total corrective $\boldsymbol{\tau} \approx [128, 62.4, -48, 9.6, 0, 0, 0]^\top$ Nm (plus full gravity terms). This rapidly drives error to zero while compensating for the robot's higher inertial and gravitational loads.

5.2 Non-Centralized (Decentralized) Controller: Independent Joint PID Control

Each joint is controlled independently using local PID control, with optional gravity pre-compensation. This approach is robust to model uncertainties and suitable for redundant manipulators like the MAiRA Neura, where joint coupling is partially ignored for simplicity.

5.2.1 Control Law (per joint i)

$$\tau_i = K_{P,i}e_i + K_{I,i}\int e_i\,dt + K_{D,i}\dot{e}_i + \hat{G}_i(\mathbf{q})$$

where $\hat{G}_i(\mathbf{q})$ estimated gravity torque for joint i (computed from partial model or lookup table).

5.2.2 Gain Calculation Example (Empirical Starting Values for MAiRA Neura)

Table 3: Empirical PID Gains for MAiRA Neura Joints (scaled for higher payload 9–18 kg, longer reach 1.4 m, and estimated higher inertias; based on similar large 7-DOF cobots)

Joint	K_P (N·m/rad)	K_I (N·m/rad/s)	K_D (N·m·s/rad)	Rationale
1 (Base rotation)	500	80	30	High base inertia, fast yaw
2 (Shoulder pitch)	800	120	50	Heavy gravity load, primary
3 (Upper arm)	600	100	40	Significant arm mass and le
4 (Redundant elbow)	400	70	30	Redundancy joint, moderat
5 (Wrist pitch/yaw)	200	40	15	Lighter distal segmen
6 (Wrist roll 1)	150	30	10	Reduced inertia
7 (Wrist roll 2/flange)	100	20	8	Lowest inertia, pure tool ro

5.2.3 Numerical Example (Joint 2 – Shoulder Pitch)

$e_2 = -0.05$ rad, $\dot{e}_2 = -0.1$ rad s^1, accumulated integral $\int e_2\,dt = 0.03$ s·rad:

Proportional: $800 \times (-0.05) = -40$ Nm Integral: $120 \times 0.03 = 3.6$ Nm Derivative: $50 \times (-0.1) = -5$ Nm Gravity estimate $\hat{G}_2 \approx 100$ Nm (higher counteracting droop due to extended arm and payload capacity)

Total $\tau_2 \approx -40 + 3.6 - 5 + 100 \approx 58.6$ Nm

This corrects position while compensating significant gravitational effects independently.

5.2.4 Tuning Notes

- Start with low integral (I) and derivative (D) terms to avoid instability or oscillations, especially in proximal joints with high inertia.
- For safety in collaborative mode (human-robot interaction), limit proportional gains K_P to reduce effective stiffness (e.g., use 50–70% of values above) and incorporate force-torque limiting from the robot's integrated sensors.
- Add velocity and acceleration feedforward where possible: $+\hat{M}_{ii}(\mathbf{q})\ddot{q}_{d,i} + \hat{C}_{ii}(\mathbf{q}, \dot{\mathbf{q}})\dot{q}_{d,i}$ for smoother tracking of dynamic trajectories.
- Exploit redundancy by adding a low-gain null-space term (e.g., joint centering) in a hybrid scheme if needed.
- Decentralized PID is simpler to implement and more robust to model inaccuracies than full CTC, making it ideal for initial deployment and real-time safety on the MAiRA Neura.

Both controllers ensure stable tracking of operational space trajectories (e.g., scaled triangular path in larger workspace). Centralized CTC offers superior precision (error ¡ 0.5 mm with good model) but requires accurate dynamics; decentralized PID is simpler, more robust, and suitable for collaborative applications with integrated cognitive sensing on the MAiRA Neura. Validate gains experimentally with low-speed, low-payload trajectories first, leveraging the robot's high repeatability and force sensing.